FPT Education
FPT UNIVERSITY

Chapter 11: BusyBox

FPT Education

FPT UNIVERSITY

Introduction to BusyBox

BusyBox gom nhiều lệnh Linux thông dụng vào một chương trình nhỏ, giúp xây dựng hệ Linux nhúng gọn nhẹ

- Replaces many standard utilities
- Single binary reduces system size
- Common in embedded Linux

Feature	BusyBox	GNU Coreutils + tools
Binary size	~1 MB	Hundreds of MB
Applets/tools	300+ in 1 binary	Hundreds of separate binaries
Dependencies	Minimal	Many libraries
Use case	Embedded systems	Desktop / server

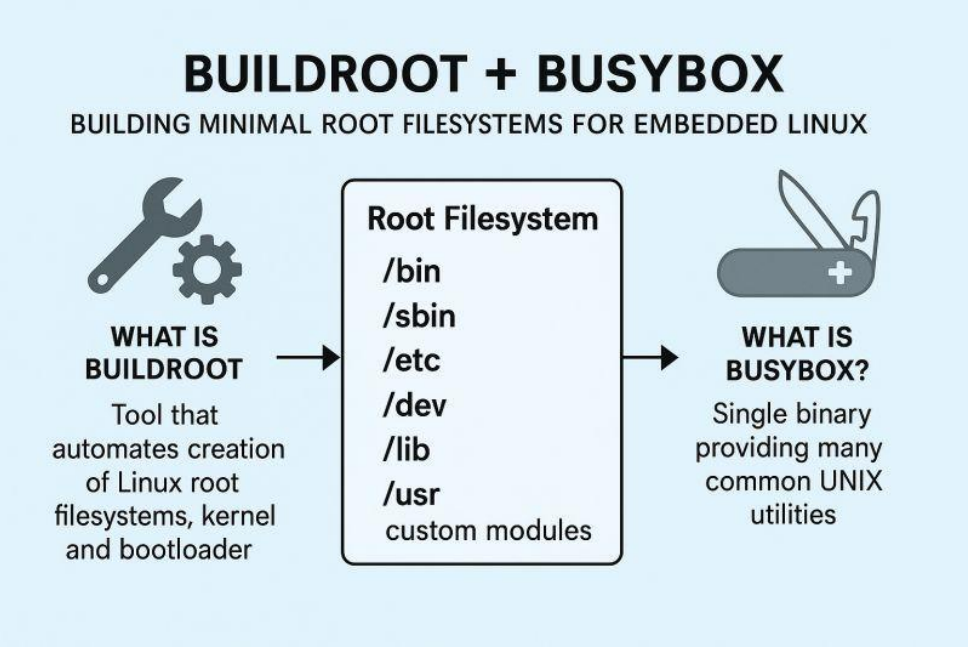
FPT Education
FPT UNIVERSITY

BusyBox Is Easy

- Configure → Build → Install → RootFS
- Similar to Linux kernel build
- Generates binary and symlinks

Symlink (symbolic link) là một file đặc biệt chứa đường dẫn trỏ đến file hoặc thư mục khác, gần giống shortcut trên Windows.

 Minimal setup makes BusyBox ideal for rapid prototyping



FPT Education

FPT UNIVERSITY

BusyBox Workflow

1) Configure

make menuconfig

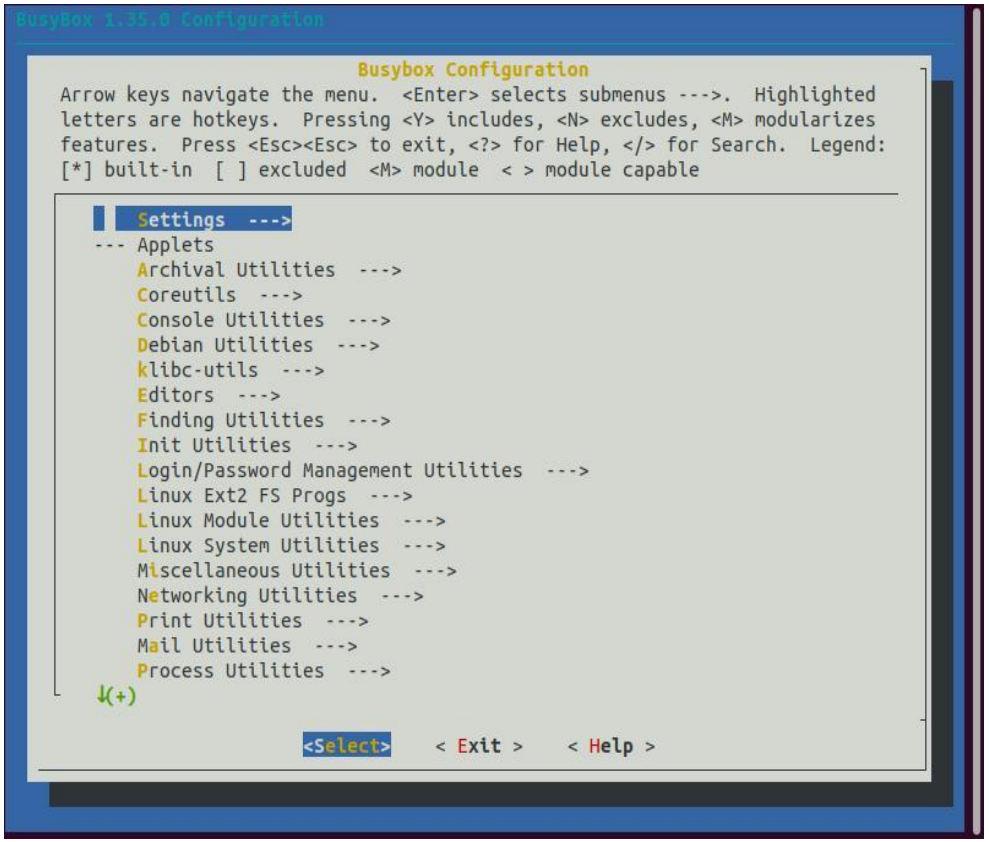
2) Build

make

3) Install to rootfs

make install

- menuconfig → select required applets
- make → build single busybox binary
- make install → create symlinks (ls, cp, sh, …)



FPT Education
FPT UNIVERSITY

BusyBox Configuration

- Configuration Interface
  - Use make menuconfig to enable or disable individual applets and customize BusyBox features.

```
# 1) Configure (select applets, enable static if needed)
make menuconfig
# 2) Build (example: cross-compile for ARM)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
# 3) Install into rootfs (creates /bin/busybox + symlinks)
make CONFIG_PREFIX=$PWD/rootfs install
```

binary BusyBox đã biên dịch và các symlink vào thư mục rootfs

- Static vs Dynamic Build
  - Static builds include all libraries in one binary, while dynamic builds rely on shared libraries (smaller but dependent).
- Applet Selection
  - Only required utilities are included, minimizing size and optimizing embedded system resources.

	Static build	Dynamic build
Mã thư viện cần thiết	Được liên kết vào binary	Nằm trong các thư viện dùng chung
Kích thước binary	Thường lớn hơ	Thường nhỏ hơn
Rootfs cần thêm thư viện dùng chung	Không cần cho binary này	Có, cùng bộ nạp động phù hợp
Triển khai rootfs tối giản	Dễ hơn	Phải chuẩn bị đủ thành phần phụ thuộc

FPT Education
FPT UNIVERSITY

Cross-Compiling BusyBox

- Cross Toolchain Usage
  - BusyBox is built using a cross-compilation toolchain targeting the embedded architecture (e.g., ARM, MIPS).

```
# Build for ARM64 (aarch64)
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
# Build for RISC-V (64-bit)
make ARCH=riscv CROSS_COMPILE=riscv64-linux-gnu-
# Install into rootfs (creates busybox + symlinks)
make CONFIG_PREFIX=$PWD/rootfs install
```

- Compiler Prefix Setup
  - Set the CROSS_COMPILE variable to specify the correct toolchain prefix (e.g., arm-linux-gnueabihf-).
- Embedded Workflow
  - Standard process for building software on a host machine for execution on a different target platform.

máy tính x86 biên dịch → tạo BusyBox cho ARM64 → đưa vào rootfs → chạy trên thiết bị ARM64.

FPT Education

FPT UNIVERSITY

Multicall Binary Concept

Multicall binary là một file thực thi duy nhất nhưng có thể thực hiện nhiều chức năng, tùy theo tên được dùng để gọi nó. BusyBox hoạt động theo cơ chế này.

- Single binary /bin/busybox

  argv[0] thường chứa tên hoặc đường dẫn được dùng để gọi chương trình.
- Behavior depends on invoked name (argv[0])
- Commands via symlinks (ls, cat, etc.)

```
# ls -lh /bin/
total 681
lrwxrwxrwx    1 root     root           7 Jan 22  2019 arch -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 ash -> busybox
-rwxr-xr-x    1 sshd     sshd      680.2K Jan 25  2019 busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 cat -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 chattr -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 chgrp -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 chmod -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 chown -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 cp -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 cpio -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 date -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 dd -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 df -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 dmesg -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 dnsdomainname -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 dumpkmap -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 echo -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 egrep -> busybox
lrwxrwxrwx    1 root     root           7 Jan 22  2019 false -> busybox
```

VD: cat hello.txt
Quá trình diễn ra:
1. Shell tìm lệnh cat, chẳng hạn tại /bin/cat.
2. /bin/cat là symlink nên hệ thống thực thi /bin/busybox.
3. Tên gọi trong argv[0] vẫn là cat hoặc đường dẫn kết thúc bằng cat.
4. BusyBox nhận ra tên cat và chạy applet đọc, in nội dung file hello.txt.

busybox:

```
argc    = 2;
argv[0] = "cat";        // Tên gọi chương trình
argv[1] = "hello.txt";  // Đối số thứ nhất

if (strcmp(argv[0], "cat") == 0)
    cat_main(argc, argv);
else if (strcmp(argv[0], "ls") == 0)
    ls_main(argc, argv);
```

argv[0] không được ghi cố định là "cat" trong source code;
giá trị này được truyền vào khi chạy chương trình.
Thực tế nó có thể chứa cả đường dẫn như /bin/cat,
nên BusyBox lấy phần tên cuối để nhận diện lệnh.

FPT Education

FPT UNIVERSITY

BusyBox init

BusyBox init là chương trình khởi tạo hệ thống tối giản, chạy với PID 1,
có nhiệm vụ chạy các bước thiết lập ban đầu và khởi động những tiến trình được cấu hình.

- BusyBox provides a minimal init system (PID 1)
- Uses /etc/inittab for system startup configuration
- Simpler than SysV, much smaller than systemd
- BusyBox /etc/inittab Format
  - <id>:<runlevels>:<action>:<process>
    - ::sysinit:/etc/init.d/rcS
    - ::askfirst:/bin/sh
    - ::respawn:/sbin/getty -L ttyS0 115200 vt100
    - ::ctrlaltdel:/sbin/reboot
- Lightweight init system

:: nghĩa là hai trường đầu để trống.

thường chứa lệnh mount filesystem, thiết lập mạng hoặc khởi tạo thiết bị.

Hiển thị yêu cầu nhấn Enter, rồi mới mở shell /bin/sh trên console.
Khi shell kết thúc, init quay lại chờ người dùng nhấn Enter để mở lại

Ctrl+Alt+Del, chạy /sbin/reboot để khởi động lại hệ thống

respawn: Tự chạy lại tiến trình nếu nó kết thúc
/sbin/getty: Chuẩn bị terminal và bắt đầu quá trình đăng nhập
-L: Dùng đường truyền cục bộ, không yêu cầu tín hiệu carrier của modem
ttyS0: Cổng serial được sử dụng
115200: Tốc độ baud
vt100: Kiểu terminal

id: Xác định terminal dùng cho tiến trình;
để trống thường dùng console
runlevels: BusyBox init bỏ qua trường này
action: Quy định khi nào và cách chạy tiến trình
process: Lệnh hoặc script cần chạy

FPT Education
FPT UNIVERSITY

rcS Initialization Script

- Executes /etc/init.d/rcS

```
#!/bin/sh
# Mount pseudo filesystems (required early in boot)
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs devtmpfs /dev
# Optional: mount tmpfs for runtime data
mount -t tmpfs tmpfs /tmp
# Print message
echo "System initialized"
# Start a shell (for debugging)
exec /bin/sh
```

script được thực thi bằng shell /bin/sh.

-t chỉ định loại filesystem.
none, devtmpfs ở vị trí nguồn chỉ là tên đại diện trong các ví dụ này, không phải ổ đĩa.

Mở shell để debug

xec thay thế tiến trình đang chạy script bằng shell mới, giữ nguyên PID của tiến trình đó.

- Mounts /proc and /sys

đoạn script minh họa hiện tại chỉ mount filesystem, in thông

- Starts services

FPT Education

FPT UNIVERSITY

BusyBox Installation

cách đưa BusyBox đã biên dịch vào rootfs để triển khai lên thiết bị đích.

- Chép binary busybox vào thư mục đích.
- Tạo các symlink cho những applet đã bật trong cấu hình, như ls, cp, sh, init.

- make install creates symlinks
- Use CONFIG_PREFIX for rootfs
  - # Install BusyBox into target root filesystem
  - make CONFIG_PREFIX=/path/to/rootfs install

- Deploy to target system

```
/path/to/rootfs/
├── bin/
│   ├── busybox
│   ├── ls -> busybox
│   ├── cp -> busybox
│   └── sh -> busybox
├── sbin/
│   ├── init -> busybox
│   └── ifconfig -> busybox
├── usr/bin/
│   └── [optional applets]
└── usr/sbin/
    └── [optional applets]
```

FPT Education
FPT UNIVERSITY

BusyBox Applets

- Hundreds of commands supported
- Reduced feature set
- Use --help for options

- Core utilities (coreutils-like):
  - ls, cp, mv, rm, cat, chmod, chown, echo
- Shell utilities:
  - sh (ash), test, true, false
- System/init tools:
  - init, halt, reboot, dmesg, mount, umount
- Networking:
  - ifconfig, ip, route, ping, netcat (nc), wget
- File & text processing:
  - grep, sed, awk, sort, find
- Editors & misc:
  - vi, less, watch, top

FPT Education

FPT UNIVERSITY

Introduction to runit

runit là hệ thống giám sát dịch vụ nhỏ gọn: khởi chạy dịch vụ, theo dõi tiến trình và tự chạy lại nếu tiến trình kết thúc.

- runit is a lightweight service supervision system
- No built-in dependency resolution (no dependency graph)
- Services are started independently by runsvdir
- Each service is supervised (auto-restart if it exits)

runsvdir: Theo dõi thư mục chứa các dịch vụ và tạo một tiến trình runsv cho mỗi dịch vụ.

FPT Education

FPT UNIVERSITY

Getting runit

hai cách đưa các công cụ giám sát dịch vụ kiểu runit vào rootfs

- Buildroot Integration
  - runit is enabled through Buildroot configuration, allowing automatic inclusion in the root filesystem.
  - make busybox-menuconfig mở giao diện cấu hình BusyBox mà Buildroot sử dụng
  - make
- Configure BusyBox directly inside BusyBox source tree
  - make menuconfig
  - make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-
  - make CONFIG_PREFIX=/path/to/rootfs install
- Enable runit Utilities
  - Select runit packages in busybox-menuconfig or Buildroot package settings.
- Service Directory
  - Default service directory is /etc/sv, where each service is defined and managed.

/etc/sv là nơi thường dùng để lưu định nghĩa dịch vụ,
không phải đường dẫn mặc định bắt buộc cho mọi hệ thống.

FPT Education
FPT UNIVERSITY

BusyBox runit

- Here is a sample `run` script for an `sshd` service using BusyBox runit:

```
#!/bin/sh
```
chọn shell thực thi

```
# Will generate host keys if they don't already exist
```
Tạo khóa nhận diện SSH server

```
ssh-keygen -A >/dev/null 2>&1
```
Đọc cấu hình bổ sung nếu có

```
[ -r conf ] && . ./conf
```
[-r conf]: Kiểm tra file conf có tồn tại và đọc được không
&&: Chỉ thực hiện bên phải nếu kiểm tra bên trái thành công
. ./conf: Thực thi nội dung conf trong shell hiện tại
Dấu chấm . là lệnh source: các biến được khai báo trong conf sẽ dùng được ở dòng tiếp theo.

```
exec /usr/sbin/sshd -D $OPTS 2>&1
```
Chạy SSH server để runit giám sát

exec: thay thế tiến trình shell bằng sshd, giữ nguyên PID.
-D: giữ tiến trình SSH server ở foreground, không tự tách ra chạy nền;
nhờ đó runit theo dõi trực tiếp được.
$OPTS: thêm các tùy chọn đã nạp từ conf.
2>&1: gộp đầu ra lỗi vào đầu ra thông thường; ở dòng này không có /dev/null
nên không có nghĩa là bỏ đầu ra.

FPT Education
FPT UNIVERSITY

Service Structure

- Service Isolation
  - Each service is defined in its own directory, enabling modular and independent management.
- run Script (Required)
  - An executable script that starts and maintains the service process.
- Optional Scripts
  - log/run → handles logging
  - finish → executes on service termination

```
/path/to/rootfs/
├── bin/
│   ├── busybox
│   ├── ls -> busybox
│   ├── cp -> busybox
│   ├── sh -> busybox
│   └── ...
├── sbin/
│   ├── init -> busybox
│   ├── mount -> busybox
│   └── ...
├── usr/
│   ├── bin/
│   │   └── [optional applets]
│   └── sbin/
│       └── [optional applets]
├── linuxrc -> bin/busybox
```

FPT Education
FPT UNIVERSITY

Service Supervision

- runsvdir (Service Manager)
  - Continuously scans service directories and launches a runsv process for each service.
- runsv (Service Supervisor)
  - Monitors the service process and ensures it is always running.
- Automatic Restart
  - If a service crashes or exits unexpectedly, it is automatically restarted.



FPT Education
FPT UNIVERSITY

Service Dependencies

- Manual Dependency Handling
  - runit does not provide a built-in dependency system; dependencies must be managed explicitly in scripts.
- Using sv and Checks
  - Services can wait for others using sv start, sv check, or custom shell logic.
- Flexible Control
  - Developers can implement custom dependency logic based on system requirements.

```
sv start: Yêu cầu khởi chạy dịch vụ và chờ kết quả trong thời hạn
sv check: Chờ dịch vụ đạt trạng thái được yêu cầu; nếu có script check,
dùng nó để kiểm tra thêm
Logic shell tự viết: Kiểm tra điều kiện cụ thể: kết nối database,
có địa chỉ IP, thiết bị đã xuất hiện…
```

FPT Education

FPT UNIVERSITY

Service Logging

thay vì service tự ghi trực tiếp vào file log, output của service được chuyển sang một process logging riêng là svlogd.

- svlogd Logging Tool
  - Logging is handled by svlogd, a lightweight and efficient logging utility designed for runit.
    - #!/bin/sh
    - exec svlogd -tt /var/log/web

#!/bin/sh: script được chạy bằng shell /bin/sh.
svlogd: chương trình nhận log và ghi xuống file.
-tt: thêm timestamp dễ đọc vào mỗi dòng log.
/var/log/web: thư mục chứa log của service web.
exec: thay process shell hiện tại bằng svlogd.

- Separate Logging Process

service và logger là hai process riêng

  - Each service can have its own dedicated logging process, improving reliability and isolation.
- Automatic Log Rotation
  - Logs are automatically rotated based on size and number of files, preventing storage overflow.

Khi đạt điều kiện cấu hình, file hiện tại được đóng và chuyển thành file log cũ, sau đó một current mới được tạo. Tránh kích thước file tăng mãi.

FPT Education

FPT UNIVERSITY

Service Control

sv không trực tiếp start/stop chương trình. sv gửi lệnh tới runsv – supervisor đang quản lý service – thông qua FIFO supervise/control.

- sv Command Interface
  - Services are managed using the sv tool with commands like up, down, status, and restart.
- Control via FIFO
  - sv communicates with the service supervisor by writing control signals to a FIFO (supervise/control).
- Signal-Based Actions
  - Control characters are translated into POSIX signals to start, stop, or manage services.

sv gửi control command,
sau đó runsv xử lý command đó.

Start (bring service up)
sv up /etc/sv/sshd

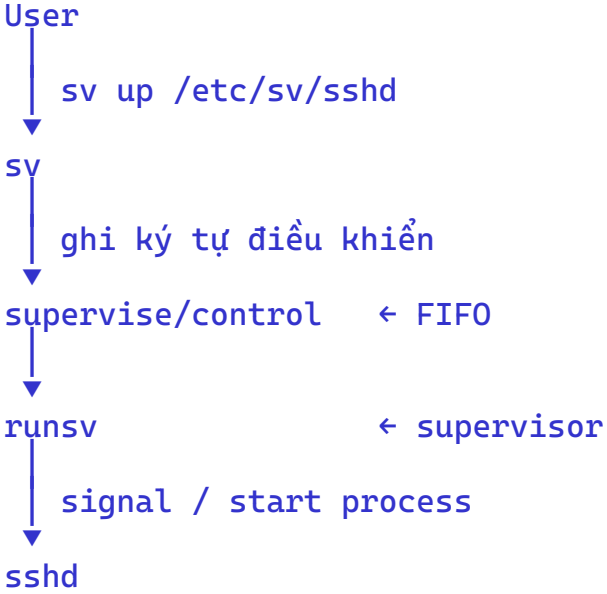
Stop (bring service down)
sv down /etc/sv/sshd

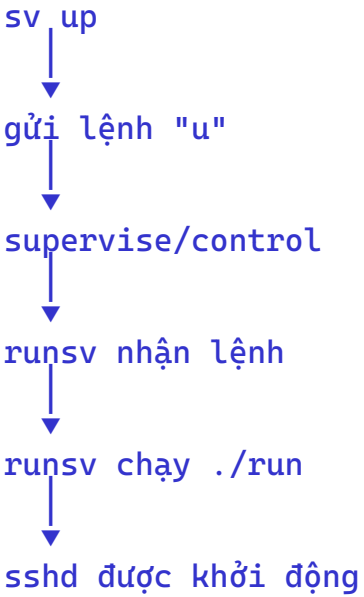
Check status
sv status /etc/sv/sshd

Restart service
sv restart /etc/sv/sshd

sv = client điều khiển
runsv = supervisor thực sự quản lý process

sv → gửi command cho runsv
runsv → gửi signal cho service

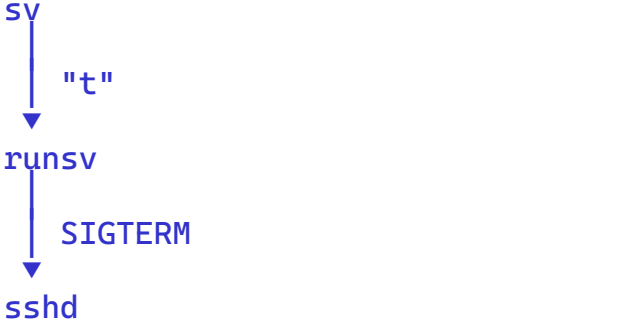




Một số control command thường gặp trong runit gồm:

```
u   → up
d   → down
t   → terminate
h   → hangup
i   → interrupt
k   → kill
```

Ví dụ khi muốn terminate process:



Signals and Behavior

- Control Characters → Signals
  - Commands sent via sv are translated into single-character codes that map directly to POSIX signals.
- Standard Signals
  - Examples:
    - u → start service
    - d → stop service
    - t → terminate (SIGTERM)
- Custom Signals
  - Supports USR1 and USR2 for application-specific behaviors.

FPT Education
FPT UNIVERSITY

Summary

- BusyBox reduces rootfs size
- Multicall binary design
- Supervision ensures reliability
- Essential for embedded Linux